

EXPERIMENT 4: Implement AO* Search algorithm

EVALUATION

Writeup (3M)	
Execution (5M)	
Viva Voce (2M)	
Total Marks (10M)	
Faculty Signature and Date	

4.1. AIM

To implement the AO* (And-Or Star) Search Algorithm to find the optimal solution graph in an AND-OR graph.

4.2. DESCRIPTION

AND-OR Graphs

An AND–OR graph is a type of graph used in Artificial Intelligence search problems where solving a problem may require **solving multiple sub-problems together (AND)** or **choosing one among several alternatives (OR)**.

In normal graphs (like in A*), reaching **one goal node** solves the problem.

In an AND–OR graph, a node can be solved in two ways:

i. OR Node

Only one of the child nodes needs to be solved.

Represents alternative choices.

Example: If you want to travel from city A to B: Go by train **OR** bus **OR** flight

Only one option is needed.

ii. AND Node

All child nodes must be solved.

Represents subtasks that must all be completed.

Example:

To cook rice: Boil water **AND** add rice **AND** cook

All steps are required.

The AO* algorithm is an informed search algorithm used for solving problems represented as **AND-OR graphs**.

Unlike A* (which works on OR graphs), AO* works when:

- Some nodes require solving **all children** (AND nodes).
- Some nodes require solving **any one** of the children (OR nodes).

AO*:

- Uses **heuristic values**.
- Expands the most promising node.
- Updates cost values dynamically.
- Marks nodes as **SOLVED** when optimal substructure is found.
- Backtracks to update parent costs.

It is widely used in:

- Problem reduction systems
- Theorem proving
- Game trees
- Planning systems

4.3. ASSUMPTIONS AND CONSTRAINTS

Assumptions:

1. The graph is a **directed acyclic graph (DAG)**.
2. Heuristic values are given for all nodes.
3. Edge costs are known.
4. Goal nodes have heuristic value = 0.
5. The graph is finite.

Constraints:

1. No negative edge costs.
2. The heuristic function should be admissible for optimality.
3. Cycles are not considered in this basic implementation.

4.4. ALGORITHM

AO* Algorithm (for Explicit AND–OR Graph Representation)

Input: Graph (G) where each node has:

- **type** = AND or OR
- **children** = list of (child, cost)
- Heuristic values ($h(n)$)
- Start node (S)

Output

- Optimal solution graph
- Minimum cost from start node

Algorithm

Step 1: Initialize:

- solution = { }
- Start from node S

Step 2: Define procedure **AO_STAR(node)**

Step 3: If **node is a terminal node** (i.e., node not present in graph)
Return its heuristic value: return $h(\text{node})$

Step 4: Read the **type of node**
node_type = graph[node].type
children = graph[node].children

Case 1: OR Node

Step 5: For every child (c) of node:
cost(c) = edge_cost(node,c) + AO_STAR(c)

Step 6: Select the child with **minimum cost**
best_child = argmin(cost(c))

Step 7: Update heuristic of node
 $h(\text{node})$ = minimum cost

Step 8: Store solution
solution[node] = best_child

Step 9: Return, return $h(\text{node})$

Case 2: AND Node

Step 10: For each child (c):
cost(c) = edge_cost(node,c) + AO_STAR(c)

Step 11: Compute total cost: total_cost = $\sum \text{cost}(c)$

Step 12: Update heuristic: $h(\text{node})$ = total_cost

Step 13: Store solution, solution[node] = list of all children

Step 14: Return return $h(\text{node})$

Step 15: Print **optimal solution graph** starting from node S.

Mathematical Representation

OR Node $Cost(n) = \min_{c \in \text{children}} (cost(n, c) + h(c))$

AND Node $Cost(n) = \sum_{c \in \text{children}} (cost(n, c) + h(c))$

Time Complexity (conceptual)

Worst case: $O(b^d)$

Where: **b** = branching factor, **d** = depth of graph

However AO* typically explores **fewer nodes** due to heuristics.

4.5. PROGRAM

<https://github.com/mayurjambit/BAD402>

```
# Graph with explicit node types
graph = {
    'A': {'type': 'OR', 'children': [('B',1), ('C',1)]},
    'B': {'type': 'AND', 'children': [('D',1), ('E',1)]},
    'C': {'type': 'OR', 'children': [('F',1), ('G',1)]}
}

# Heuristic values
heuristic = {
    'A': 6,
    'B': 4,
    'C': 3,
    'D': 0,
    'E': 0,
    'F': 0,
    'G': 0
}

graph = {
    'A': {'type':'OR', 'children':[('B',1), ('C',1)]},
    'B': {'type':'AND', 'children':[('D',2), ('E',2)]},
    'C': {'type':'OR', 'children':[('F',2), ('G',3)]},
    'D': {'type':'OR', 'children':[('H',1), ('I',2)]},
    'E': {'type':'AND', 'children':[('J',1), ('K',1)]},
    'F': {'type':'OR', 'children':[('L',3)]}
}

heuristic = {
    'A':10,
    'B':8,
    'C':6,
    'D':4,
    'E':5,
    'F':4,
    'G':0,
    'H':0,
    'I':0,
    'J':0,
    'K':0,
    'L':0
}

solution = {}
def ao_star(node):
    if node not in graph:
        return heuristic[node]
    node_type = graph[node]['type']
    children = graph[node]['children']
    # OR NODE
```

```
if node_type == "OR":
    min_cost = float('inf')
    best_child = None
    for child, cost in children:
        total_cost = cost + ao_star(child)
        if total_cost < min_cost:
            min_cost = total_cost
            best_child = child
    heuristic[node] = min_cost
    solution[node] = [best_child]
    return min_cost

# AND NODE
if node_type == "AND":
    total_cost = 0
    best_children = []
    for child, cost in children:
        total_cost += cost + ao_star(child)
        best_children.append(child)
    heuristic[node] = total_cost
    solution[node] = best_children
    return total_cost

# Function to print only optimal solution graph and the optimal cost
def print_solution(node):
    optimal_cost = 0
    print(node)
    if node in solution:
        for child in solution[node]:
            optimal_cost += heuristic[child]
            print_solution(child)

# Run AO*
total_cost = ao_star('A')
print("Optimal Solution Graph:")
print_solution('A')
print("Optimal Cost:", total_cost)
```

4.6. OUTPUT